

Lab 1

Pointers & Dynamic Memory Allocation

In this lab, we will review the techniques for C++ programming with pointers, string, linked lists, stacks, and queues.

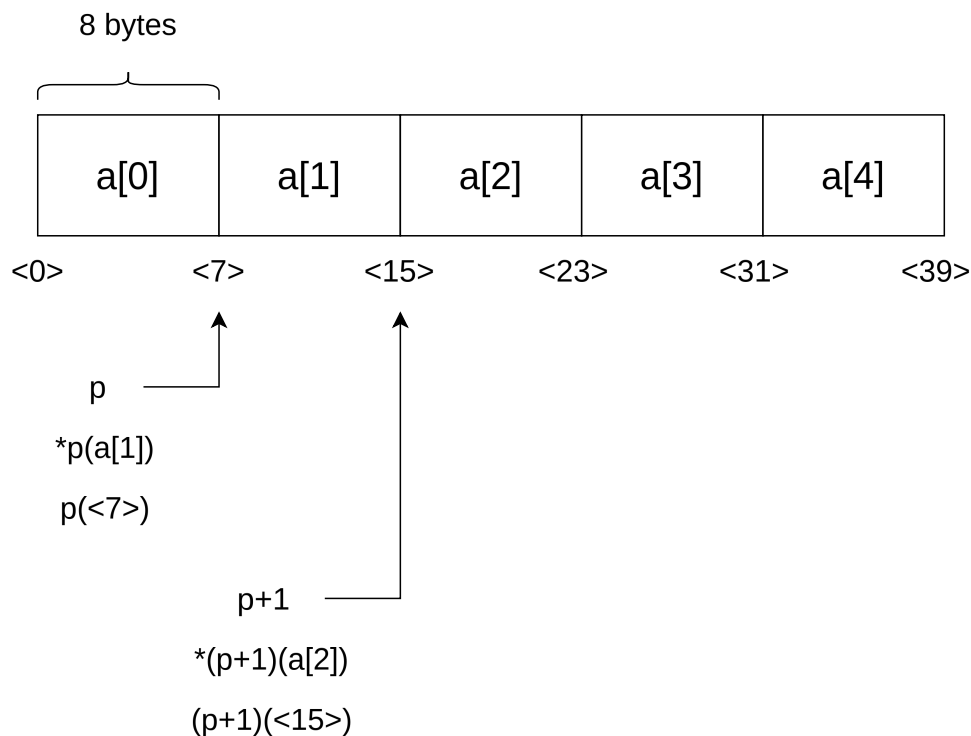
1 What's pointers?

A pointer variable is a memory address. For example, `int* p` is a pointer variable named `p`, and the value of `p` is the address of an ordinary `int`-type variable.

Consider the following example:

```
1 double a[5];  
2 double *p;  
3 p = &a[1];
```

Since a double-type variable requires 8 bytes of storage, we have an array with 5 elements, each using 8 bytes, resulting in a total of 40 bytes needed.



We can see that pointer arithmetic has some differences. For example, if `p` is pointing to address `<7>`, then `(p+1)` will point to `<15>`, meaning `p + 1` translates to `p + 1 × sizeof(double) → <7> + 8 = <15>`. When working with pointers, we need to understand the following basic notations: `p` is the address that `p` is pointing to—in this case, `<7>`; `*p` is the value at the memory location `p` is pointing to—in this case, `a[1]`; and finally, `&p` is the address of the pointer `p` itself (the diagram does not show the address of the pointer `p`). If we want to access the element `a[0]`, we need the code `*(p - 1)`.

2 Heap and Stack Memory

We note that when using pointer variables, we often request memory allocation. For example, `int* p = new int[5]` means that the `int`-type pointer `p` requests the allocation of 5 `int`-type elements in the Heap memory region. After allocation, we use it for a specific purpose, and when we're done, we need the code `delete[] p` to deallocate the allocated elements. If this line of code is missing, the Heap memory will still hold the 5 elements, and this phenomenon is called a memory leak.

In the example from Section 1, we see that the pointer `p` receives an address from a statically declared array in the Stack, so we do not need to use the `delete` statement to free the memory in the Stack.

3 Illustrative Examples

In this section, we will be introduced to code snippets that work with pointer variables and one-dimensional arrays. The code includes functions for inputting an array, outputting an array, and inserting an element into the array.

Here is the C++ code demo:

```
1 #include <malloc.h>
2 #include <iostream>
3 using namespace std;
4
5 void OutputRealArray(float a[], int n) {
6     for(int i = 0; i < n; i++)
7         cout << a[i] << " ";
8 }
9
10 void InsertElement(float*& a, int& n, float x) {
```

```
11     int m = n + 1;
12     float* anew = (float*)realloc(a, m * sizeof(float));
13     if(anew != NULL) {
14         anew[n] = x; n++;
15         a = anew;
16     }
17 }
18
19 void InputRealArray(float*& a, int& n) {
20     float x;
21     a = NULL; n = 0;
22     while(cin >> x) {
23         InsertElement(a, n, x);
24     }
25 }
26
27 int main() {
28     float* B; int nB;
29     cout << "Enter real numbers into the array: " << endl;
30     InputRealArray(B, nB);
31     cout << nB << " elements: " << endl;
32     OutputRealArray(B, nB);
33     if(B != NULL) {
34         delete[] B;
35     }
36     return 0;
37 }
```

We will examine each function in detail. The **OutputRealArray** function takes a real number array and the number of elements as input, with no output. Its primary function is to print the elements of the array to the screen. The **InsertElement** function takes a real number array, the number of elements, and the element to be inserted as input. Note the ***&** notation for the first parameter, where we use a pointer reference. Additionally, we use the **realloc** function to change the size of the array. We declare an additional array **anew** because the **realloc** function might delete the old array **a** to create a new one; for safety, we declare **anew** and then assign it back to **a**. The final function, **InputRealArray**, takes a real number array and the number of elements as input. In this function, we call the **InsertElement** function to successively add elements to the array.

4 Structured Pointers

The concept of structures is introduced to extend basic data types. Consider the Fraction type, defined as follows:

```
1 struct Fraction {
2     int numerator, denominator;
3 };
4 typedef struct Fraction Fraction;
```

To perform input and output operations with the Fraction type, such as `cin >> fractionPtr` or `cout << fractionPtr`, we need to overload these operators for the Fraction type.

Here's the implementation of the overloaded operators:

```
1 #include <iostream>
2 using namespace std;
3
4 struct Fraction {
5     int numerator, denominator;
6 };
7 typedef struct Fraction Fraction;
8
9 ostream& operator<< (ostream& output, const Fraction& fraction) {
10     if (fraction.denominator == 1 || fraction.numerator == 0)
11     {
12         output << fraction.numerator;
13     }
14     else
15     {
16         output << fraction.numerator << "/" << fraction.denominator;
17     }
18     return output;
19 }
20
21 istream& operator>> (istream& input, Fraction& fraction) {
22     input >> fraction.numerator >> fraction.denominator;
23     return input;
24 }
25
26 int main() {
27     Fraction* fractionPtr = new Fraction;
28     cin >> (*fractionPtr);
29     cout << (*fractionPtr);
```

```
30 delete fractionPtr; // Added to prevent memory leak
31 return 0;
32 }
```

We can see that the input operator >> requires an input device (of type istream) and a reference variable of type Fraction (which is why we use the notation *fractionPtr). The same applies to the output operator <<. Additionally, to access the elements within the Fraction type, we use the -> notation. For example, fractionPtr->numerator or fractionPtr->denominator.

5 Exercise 1: Simple Pointers

Write a program with the following requirements:

1. **Input Array:** Write a function to input an array from the keyboard with a known size.

```
1 void inputArray(int* &arr, int n);
```

2. **Print Array:** Write a function to print the elements of the array to console.

```
1 void printArray(int* arr, int n);
```

3. **Find Maximum Value:** Write a function to find the maximum value in the array.

```
1 int findMax(int* arr, int n);
```

4. **Sum of array:** Write a function to calculate the sum of the elements in the array.

```
1 int sumArray(int* arr, int n);
```

5. **Concatenate Arrays:** Write a function to concatenate two arrays into a new array.

```
1 void concatArrays(int* a, int na, int* b, int nb, int* &res, int &nres);
```

6. **Longest Ascending Subarray:** Write a function to find the longest ascending subarray.

```
1 void findLongestAscendingSubarray(int* arr, int n, int* &res, int &nres);
```

7. **Main function:** In the main function, perform the following tasks:

- Enter the number of elements in array a. Then enter the array a.

- Enter the number of elements in array **b**. Then enter the array **b**.
- Print the array **c**, which is the concatenation of arrays **a** and **b**.
- Print the maximum value in the array **c**.
- Print the sum of elements in the array **c**.
- Print the longest ascending subarray in the array **c**.

6 Exercise 2: Structured Pointers

Based on the examples above in Section 4, students should design an array of Fraction structures with functions that perform the following tasks:

- Input an array of fractions
- Output an array of fractions
- Insert a fraction at the end of the array
- Delete a fraction from the end of the array
- Deallocate the fraction array that was dynamically allocated

Students should design the input and output parameters for all functions, then write a `main()` function to demonstrate the functions they have created.

7 Exercise 3: Pointer to Pointer

Write a program with the following requirements:

1. **Read Matrix from File:** Write a function to read a matrix from a file.

Return `false` if the file cannot be opened or the file structure is invalid. Else, return `true`.

```
1 bool readMatrix(const char* filename, int** &matrix, int &rows, int &cols)
```

Each row of the matrix on a separate line and elements separated by spaces. For example:

```
1 2 3
```

```
4 5 6
```

2. **Print Matrix to File:** Write a function to print a matrix to a file.

```
1 void printMatrix(const char* filename, int** matrix, int rows, int cols)
```

Each row of the matrix on a separate line and elements separated by spaces. For example:

1 2 3

4 5 6

3. **Matrix Multiplication:** Write a function to multiply two matrices.

Return a boolean value indicating whether the multiplication is successful or not.

```
1 bool multiplyMatrices(int** a, int aRows, int aCols,
2                       int** b, int bRows, int bCols,
3                       int** &res, int &resRows, int &resCols);
```

4. **Matrix Transposition:** Write a function to calculate the transpose of a matrix.

```
1 void transposeMatrix(int** matrix, int rows, int cols,
2                      int** &res, int &resRows, int &resCols);
```

5. **Main Function:** In the main function, perform the following tasks:

- Read the matrix **a** from the file **matrix_a.txt**.
- Read the matrix **b** from the file **matrix_b.txt**.
- Multiply the matrices **a** and **b** to get matrix **c**. Check if the multiplication is successful.
- Print the resulting matrix **c** to the file **matrix_c.txt** if the multiplication is successful.
- Calculate the transpose of matrix **c** and print it to the file **matrix_c_transposed.txt**.

Regulations

Please follow these regulations:

- You are allowed to use any IDE.
- After completing assignment, check your submission before and after uploading to Moodle.
- Prohibited libraries: `<set>`, `<unordered_set>`, `<map>`, `<unordered_map>`, `<algorithm>`, `<list>`, `<stack>`, `<queue>`, and `<bits/stdc++.h>`.
- You can use `<vector>` or any libraries that are not in the prohibited libraries listed above.

Your source code must be contributed in the form of a compressed file and named your submission according to the format **StudentID.zip**. Here is a detail of the directory organization:

```
StudentID
├── Exercise_1.cpp
├── Exercise_2.cpp
└── Exercise_3.cpp
```

The end.